DOCUMENTO DE ARQUITETURA DO SISTEMA

Nome do Projeto:	SIGA UEA – Sistema de Gestão de TCC do NUCOMP-UEA
Instituição:	UEA – Universidade do Estado do Amazonas, Escola Superior de Tecnologia (EST) – NUCOMP
Patrocinadora / Cliente:	Profª Drª Diana Maria da Câmara – Coordenação do NUCOMP
Equipe / Gerentes do Projeto:	Davih Taumaturgo Soares, Igor Leite de Almeida, Igor Meireles de Lima Taveira, Matheus Maia Braga e Samuel Ogino Barbosa.
Documento de referência:	Plano de Integração do Projeto, versão 2.3, seções 4.6.1 e 4.6.2; história HT03 (SCRUM-108)
Data:	20/10/2026
Versão:	1.0

SUMÁRIO

1. INTRODUÇÃO **3**
1.1 Objetivo 3
1.2 Escopo 3
1.3 Documentos de referência 3
1.4 Siglas 3
2. VISÃO GERAL **3**
3. CAMADAS DO SITE **4**
4. CAMADAS DA API **4**
5. CAMINHO DE UMA REQUISIÇÃO **5**
6. SEGURANÇA **6**
7. DADOS E CONSISTÊNCIA **6**
8. AMBIENTES **6**
9. DECISÕES DE ARQUITETURA **7**
10. ATENDIMENTO AOS REQUISITOS NÃO FUNCIONAIS **8**
11. VERIFICAÇÃO DA ARQUITETURA **8**
12. REGISTRO DE ALTERAÇÕES **9**
13. APROVAÇÕES **9**
14. REFERÊNCIAS **9**

1. INTRODUÇÃO

Este documento descreve a arquitetura do SIGA UEA – Sistema de Gestão de TCC do NUCOMP-UEA: como o sistema está organizado em camadas, o caminho de uma requisição e as decisões que sustentam essa organização. Ele atende à parte de arquitetura da história HT03 (Arquitetura e contrato de API) e detalha a estratégia de integração da seção 4.6.1 do Plano de Integração do Projeto.

1.1 Objetivo

Dar à equipe uma descrição única de onde cada responsabilidade fica no código, para que o frontend e o backend evoluam em paralelo sem retrabalho e para que cada requisito novo encontre o seu lugar.

1.2 Escopo

Descreve o sistema como está na branch dev em 20/10/2026 (módulo de usuários, turmas e cadastro de TCC) e a organização que os próximos módulos seguem (defesa, publicação, indicadores e agenda).

1.3 Documentos de referência

- Plano de Integração do Projeto – SIGA UEA, versão 2.3, seções 4.6.1 (estratégia de integração) e 4.6.2 (dependências de integração).
- Contrato da API do SIGA UEA (documento que acompanha este): as 51 operações, com corpo, resposta e códigos.
- Diagramas UML de componentes, pacotes e implantação (D04, pasta 04_Arquitetura).
- Requisitos_SIGA_UEA.xlsx: requisitos, regras de negócio e requisitos não funcionais.

1.4 Siglas

Sigla	Significado
API	Interface de comunicação entre o site e o servidor (REST/JSON, sob /api/v1).
ORM	Mapeamento entre as classes do código e as tabelas do banco (SQLAlchemy).
REST	Estilo de API em que cada recurso tem um endereço e as operações usam os métodos HTTP.
RN, RNF, RBN	Regra de negócio, requisito não funcional e requisito de baixo nível da planilha.
DS-FLxx	Diagrama de sequência do fluxo FLxx (Documento de Sequência).

2. VISÃO GERAL

O SIGA UEA é uma aplicação web em três partes, que conversam somente por contratos explícitos: o site e a API integram-se pelo contrato da API; a API e o banco, pelo esquema versionado nas migrações. O serviço de e-mail fica encapsulado em um componente próprio e pode ser trocado sem alterar o restante.

Parte	Tecnologia	Papel
Site	Next.js 16 com React 19 (TypeScript, Tailwind)	Telas e navegação. Não guarda regra de negócio nem acessa o banco.
API	Python 3.12 com FastAPI, SQLAlchemy 2.1 e Alembic	Regras de negócio, autorização, validação e auditoria, expostas como REST/JSON sob /api/v1.
Banco	PostgreSQL 16 (SQLite no desenvolvimento)	Dados, restrições de integridade e o log de auditoria imutável. Esquema criado por migrações.

O sistema é um **monólito modular**: uma única API, organizada por módulo de negócio (usuários, turmas e cadastro de TCC; depois defesa, publicação e agenda), e não um conjunto de microsserviços. Para uma equipe de cinco pessoas com 8 h semanais, um único serviço para publicar, testar e manter reduz o risco de integração, apontado no Plano de Integração como o principal risco técnico do projeto.

O caminho completo é: navegador → site (HTTPS) → repasse de /api/v1 → API → PostgreSQL, com a API enviando e-mails pela API HTTPS da Brevo no ambiente integrado (o plano grátis do Render bloqueia SMTP) e pelo SMTP do Gmail no ambiente local.

3. CAMADAS DO SITE

O site fica nas pastas app/, components/ e lib/ do repositório, além do arquivo proxy.ts.

Camada	Onde	Responsabilidade
Páginas	app/(publico)/ e app/(sistema)/	Uma pasta por tela: acesso e sessão (TEL01), usuários e perfis, meu perfil, TCCs, cadastro, acompanhamento, orientações e turmas. As telas internas são componentes de cliente que chamam a API e mostram o resultado.
Componentes de interface	components/	Peças reutilizadas (campo, botão, modal, aviso, casca, selos de situação, cartão de turma), sem acesso à API. Garantem o mesmo visual e a acessibilidade em todas as telas.
Cliente da API e sessão	lib/api.ts, lib/sessao.tsx, lib/sessao-servidor.ts, lib/tipos.ts	api.ts é o único ponto que faz chamadas HTTP: monta a requisição, lê o erro no formato padrão e leva ao login quando a API responde 401. sessao-servidor.ts pergunta à API quem é o usuário antes de desenhar uma tela interna; tipos.ts espelha os esquemas da API.
Proxy de navegação e repasse	proxy.ts e next.config.ts	proxy.ts leva ao login quem abre uma tela interna sem o cookie de sessão. Os rewrites repassam /api/v1 para a API, de modo que o navegador fala com uma única origem.

Regra da camada: o site nunca decide permissão. Esconder um item do menu é apenas conveniência; a autorização é conferida pela API em toda rota.

4. CAMADAS DA API

A API fica em backend/app/. Cada camada só chama a de baixo: rotas → serviços → modelos → banco. Os esquemas, a segurança e os módulos transversais apoiam as rotas e os serviços.

Camada	Onde	Responsabilidade	Não faz
Rotas	routes/	Recebe a requisição, declara o perfil exigido, converte a entrada pelo esquema e chama um serviço.	Não tem regra de negócio nem consulta o banco.
Esquemas	schemas/ (Pydantic)	Formato de entrada e saída de cada rota: tipos, obrigatoriedade, tamanhos e validações de campo. Entrada inválida responde 400 com a mensagem de cada campo.	Não acessa o banco.
Segurança e sessão	deps.py, security.py	Exige sessão válida e renova os 60 minutos; recusa com 403 quem não tem o perfil; aplica a regra de senha, o hash bcrypt (custo ≥ 10) e os tokens de link.	Não conhece as telas.
Serviços	services/	Regras de negócio (RN04, RN05, RN07, elegibilidade do orientador, protocolo, situações). Cada operação é uma transação e grava o log de auditoria. Os comentários citam o passo do diagrama de sequência.	Não monta resposta HTTP: sinaliza o erro com ErroAPI(código, mensagem).
Modelos	models.py (SQLAlchemy)	Tabelas, relacionamentos e restrições, com os nomes do modelo lógico.	Não tem regra de negócio.
Acesso ao banco e esquema	database.py, migrations/, seed.py	Uma sessão por requisição, fechada assim que a rota termina; migrações versionadas 0001 a 0004; carga inicial repetível.	—

Módulos transversais: erros.py dá o formato único de erro; config.py lê as variáveis de ambiente; services/auditoria.py grava o log que o banco não deixa alterar (RNF03); services/email.py registra cada envio em notificacao_email e envia depois da resposta, sem atrasar quem fez o pedido. A pasta repositories/ está reservada: hoje os serviços usam o ORM diretamente.

5. CAMINHO DE UMA REQUISIÇÃO

Exemplo do envio do cadastro de TCC para análise, POST /api/v1/tccs/{id}/submissao (DS-FL01, RBN03.03 e RBN03.04):

1. No passo 3 de TEL05, o aluno confirma o envio. lib/api.ts faz a chamada ao próprio endereço do site, levando o cookie de sessão.
2. O repasse do Next.js entrega a chamada à API.
3. A rota confere a sessão (401 se inválida) e o perfil de aluno (403) e chama o serviço de submissão.
4. O serviço carrega o TCC travando a linha (SELECT … FOR UPDATE), para que dois cliques não gerem dois protocolos (RNF06), e confere o dono (404, RN07), a matrícula (403, RN05) e as pendências (422 com campos_pendentes).
5. Se o cadastro já estava em análise, devolve o mesmo protocolo. Se não, gera TCC-AAAA-NNNNN, muda a situação para EM_ANALISE e grava o histórico e a auditoria na mesma transação, agendando o e-mail ao orientador.
6. A API responde 200 com o protocolo; a sessão do banco fecha e o e-mail sai em seguida, com conexão própria.
7. A tela mostra o protocolo e bloqueia a edição.

Esse caminho foi medido com 30 envios simultâneos: p95 de 0,73 s, para a meta de 3 s do RNF01 (Relatório de Execução de Testes, aba Desempenho RNF01).

6. SEGURANÇA

Ameaça ou requisito	Como a arquitetura responde
Roubo de sessão por script na página	Token só em cookie httponly, SameSite=Lax e Secure em produção (COOKIE_SEGURO=1).
Senha vazada do banco	Só o hash bcrypt (custo ≥ 10) fica gravado; tokens de link e de sessão também só como hash.
Força bruta no login	Bloqueio de 15 minutos após 3 tentativas e mensagem genérica, que não revela se o e-mail existe.
Acesso a dado de outro usuário	Autorização por perfil em toda rota e por vínculo no serviço (RN07); sem vínculo, a API responde 404, sem revelar que o registro existe.
Interceptação	HTTPS em todo o caminho (RNF02) e TLS também entre a API e o banco no ambiente integrado.
Alteração do histórico	Log de auditoria com gatilho no banco que recusa UPDATE e DELETE (migração 0003, RNF03).
Dados pessoais (LGPD)	Cada perfil vê só o que precisa (RN07), os testes usam dados fictícios e as alterações ficam auditadas. Não se presume conformidade jurídica certificada (risco R08).

7. DADOS E CONSISTÊNCIA

- Integridade garantida também pelo banco: e-mail único sem diferenciar maiúsculas, um TCC por aluno e período, protocolo único e domínios de situação com CHECK.
- Transações curtas: cada operação do serviço é uma transação, e a sessão fecha quando a rota termina, antes das tarefas em segundo plano. Sem isso, sob carga, cada requisição segurava duas conexões e o conjunto de conexões se esgotava; a correção foi feita na S03 e confirmada no teste de carga.
- Esquema só por migração: o mesmo esquema no SQLite de desenvolvimento e no PostgreSQL, e um teste falha se o modelo mudar sem a migração correspondente.

8. AMBIENTES

Ambiente	Onde roda	Uso
Local	npm run dev, uvicorn --reload e SQLite, ou Docker Compose	Desenvolvimento e testes de cada integrante (README, seção Como rodar).
Integrado e homologação	Vercel (site), Render (API), Neon (PostgreSQL) e Brevo (e-mail)	Testes de integração e validação com a coordenação (HT06). Configuração em render.yaml, .vercelignore e README.

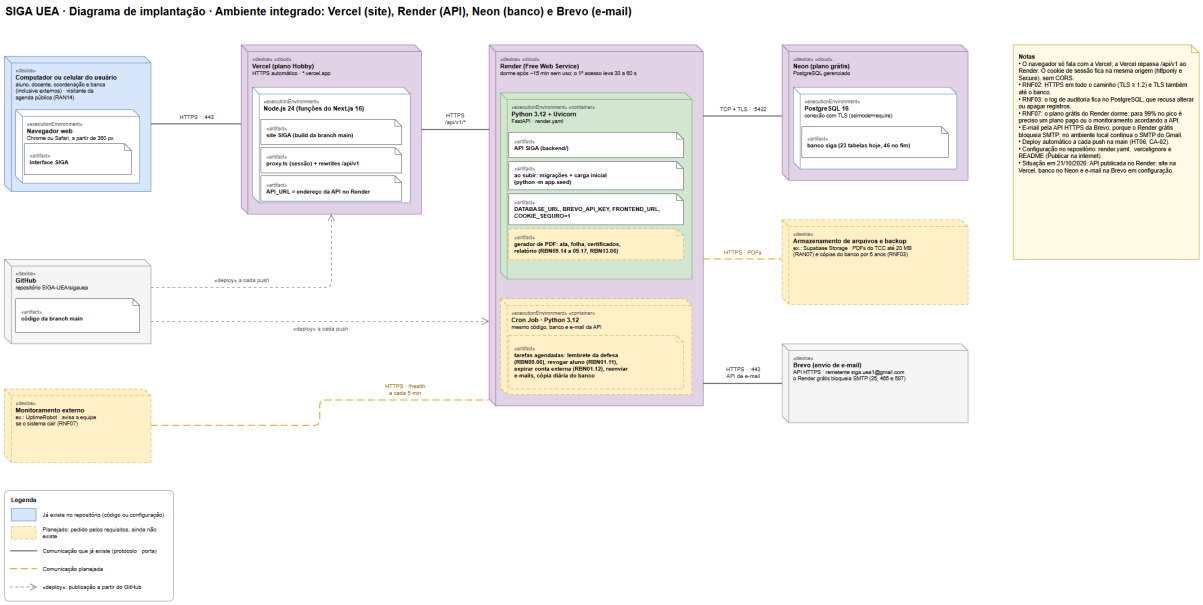


Figura 1 – Diagrama de implantação do ambiente integrado. Fonte: elaborado pelos autores (2026).

9. DECISÕES DE ARQUITETURA

Decisão	Motivo	Alternativa descartada
Monólito modular	Equipe pequena e integração como maior risco; um só serviço para publicar e testar	Microsserviços por módulo
Site e API na mesma origem, com repasse de /api/v1	Cookie de sessão sem CORS e sem expor a API diretamente	Site chamando a API em outro domínio
Sessão em cookie httponly	Protege o token de scripts na página	Token no armazenamento do navegador
Regras no serviço e rotas finas	Regras testáveis sem HTTP e reaproveitáveis	Regra na rota ou na tela
Validação responde 400; regra de negócio, 409 ou 422	É o que as fichas pedem; separa campo inválido de operação não permitida	O 422 padrão do FastAPI para tudo
Esquema por migrações	Mesmo banco em todos os ambientes e histórico no Git (HT05)	Criar as tabelas ao iniciar a aplicação
E-mail depois da resposta, registrado em notificacao_email	A falha de envio não desfaz o cadastro e pode ser reenviada (RBN01.05, RBN03.06)	Enviar dentro da requisição
E-mail pela API HTTPS da Brevo no Render	O plano grátis do Render bloqueia as portas de SMTP (25, 465 e 587); HTTPS passa em qualquer hospedagem. Sem a chave, a API usa SMTP	Gmail por SMTP direto; plano pago do Render
Restrições no banco além do código	Duplicidade e concorrência barradas mesmo com dois cliques (RNF06)	Conferir só no código

10. ATENDIMENTO AOS REQUISITOS NÃO FUNCIONAIS

A tabela compara cada requisito não funcional da planilha Requisitos_SIGA_UEA.xlsx com o que o sistema faz em 20/10/2026. Atendido: cumprido e verificado; Parcial: parte cumprida e parte pendente; Planejado: depende de um módulo ou do ambiente ainda não entregue.

RNF	O que a planilha exige	Como o sistema atende	Situação
RNF01	Submissão do cadastro em até 3 s (p95) com 30 envios simultâneos.	Trava de linha, transação curta e e-mail depois da resposta. Medido: p95 de 0,73 s com 30 envios, em PostgreSQL 16 local.	Atendido localmente; repetir no ambiente integrado
RNF02	bcrypt custo ≥ 10; HTTPS com TLS ≥ 1.2; sessão de 60 min; LGPD; agenda pública sem dados pessoais.	bcrypt custo 12, sessão de 60 min com renovação e acesso por perfil e vínculo. O HTTPS vem da Vercel e do Render. A agenda pública é do MOD07.	Parcial: HTTPS depende da publicação; agenda planejada
RNF03	Log não editável com usuário, data/hora (America/Manaus), campo, valor anterior e novo; retenção de 5 anos.	Log com todos esses campos; o banco recusa alterar ou apagar (migração 0003). A data é gravada em UTC e mostrada no fuso de Manaus.	Parcial: a retenção de 5 anos depende do backup planejado
RNF04	Sem rolagem horizontal em 360 e 768 px, no Chrome e no Safari móveis.	Telas testadas em 360 e 768 px no Google Chrome, sem rolagem horizontal (59 checagens).	Parcial: falta o teste no Safari
RNF05	Ata em PDF em até 5 s (p95) após a confirmação do resultado.	Gerador de PDF previsto na API (MOD04).	Planejado
RNF06	Sem protocolo duplicado; falha de envio não apaga a versão anterior; SHA-256 conferido no envio e na leitura.	Protocolo com trava de linha e restrição única; arquivo validado antes de substituir o anterior; SHA-256 gravado no envio e conferido ao ler (fotos). Os PDFs do TCC seguem o mesmo padrão.	Atendido para fotos e protocolo; PDFs com o RAN07
RNF07	99% de disponibilidade no pico e recuperação em até 30 min.	No plano grátis do Render a API dorme sem uso; o diagrama de implantação prevê monitoramento e backup.	Planejado: exige plano pago ou monitoramento

Pontos em que o sistema difere da planilha, por decisão registrada da equipe:

- RN16 diz que as recusas de orientação não são guardadas. O sistema guarda a recusa e o motivo, porque o RNF03 exige registrar toda alteração; a recusa não vale como vínculo. Sugere-se ajustar o texto da RN16.
- A planilha fala em e-mail institucional; por decisão da equipe, o cadastro aceita qualquer e-mail válido.
- O DS-FL01 envia o e-mail ao orientador só na submissão; como o RBN03.03 exige o aceite antes, o aviso sai quando o aluno indica o orientador, e outro sai na submissão.

11. VERIFICAÇÃO DA ARQUITETURA

- Suíte da API (backend/tests, pytest): 123 execuções em SQLite e em PostgreSQL 16, cobrindo as 51 operações do contrato.
- Testes no navegador (docs/testes/e2e): fluxos completos pela tela e responsividade em 360 e 768 px.

- Teste de carga (backend/scripts/carga_rnf01.py): RNF01 e RNF06.
- Resultados e evidências: Relatório de Execução de Testes (pasta 07_Testes).

12. REGISTRO DE ALTERAÇÕES

Versão	Data	Descrição	Responsável
1.0	20/10/2026	Criação do documento.	Igor Leite

13. APROVAÇÕES

Os campos abaixo se destinam à assinatura de quem aprova este documento. Enquanto não houver aprovação, permanecem em branco.

Assinatura – Equipe de desenvolvimento	Assinatura – Profª Drª Diana Maria da Câmara / Coordenação NUCOMP	Data

14. REFERÊNCIAS

SIGA UEA. Plano de Integração do Projeto. Versão 2.3. Manaus: NUCOMP – EST/UEA, 20 out. 2026.

SIGA UEA. Requisitos_SIGA_UEA.xlsx: matriz de requisitos, regras de negócio, requisitos não funcionais, fluxos, telas e rastreabilidade. Manaus: NUCOMP – EST/UEA, 2026.

SIGA UEA. Plano de Gerenciamento de Configuração. Versão 1.1. Manaus: NUCOMP – EST/UEA, 20 out. 2026.

SIGA UEA. Contrato da API. Versão 1.0. Manaus: NUCOMP – EST/UEA, 20 out. 2026.